Project Title: Anti-Performance Throttling Booster for Applications and Gaming Systems

Project ID: 50

Team Name: Infinite Loop

Ankita Vellara	Ashritha Reddy	Harshitha Vangaveti	Hamsa M N
PES2UG24AM021	PES2UG24AM071	PES2UG24AM064	PES2UG24AM060

SOFTWARE ARCHITECTURE

1. Description of each architectural layer

Layer	Responsibility	Suggested Implementation
Presentation	Display live metrics, alerts, profiles, and logs; accept user input	Python Tkinter
Application / Control	Coordinate monitoring, profile selection, alerts, and user-approved actions	Python controller classes
Core Processing	Collect metrics, detect suspected throttling, analyze causes, and determine safe actions	Python modules and functions
System Interface	Retrieve system statistics and communicate with process-management facilities	psutil, OS interfaces
Data / Persistence	Save profiles, timestamps, events, and optimization outcomes	JSON and/or local log files

2. Internal modules and their responsibilities

Module 1: Dashboard and User Interface

- ➢ Display CPU, memory, process, and network metrics.
- ➢ Provide Start Monitoring and Stop Monitoring controls.
- ➢ Display detected conditions, recommendations, and action results.
- ➢ Allow users to select performance profiles and view logs.

Module 2: Performance Monitoring

- ➢ Collect CPU and RAM utilization periodically.
- ➢ Retrieve running-process resource usage.
- ➢ Collect relevant network-usage statistics.
- ➢ Supply timestamped metric samples to the other modules.

Module 3: Throttling Detection and Alerts

- ➢ Evaluate metrics against configured thresholds or detection rules.
- ➢ Identify suspected throttling or resource contention.
- ➢ Generate alerts with timestamps and relevant measurements.
- ➢ Avoid treating high CPU usage alone as definitive proof of throttling.

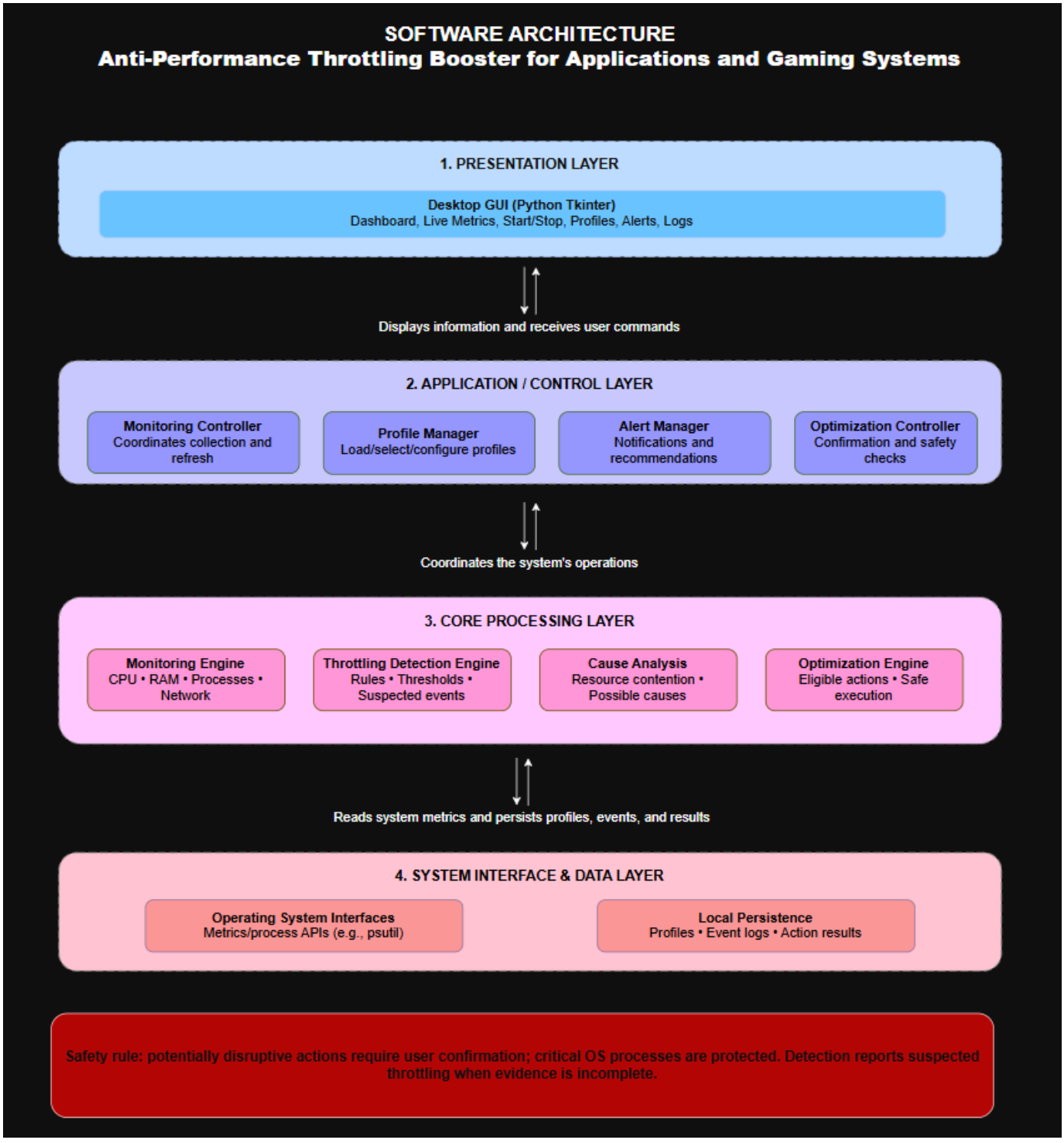
Module 4: Optimization, Profiles, and Logging

- ➢ Identify eligible background processes and possible optimization actions.
- ➢ Support user-controlled performance profiles.
- ➢ Require confirmation for potentially disruptive actions.
- ➢ Record events, recommendations, and action outcomes.

3. Mapping architecture to SRS requirements

Architecture component	Requirement IDs	What it satisfies
Monitoring engine	REQ-01 to REQ-04	Start/stop monitoring, CPU/RAM metrics, process usage, automatic refresh
Detection engine	REQ-05 to REQ-08	Detection rules, suspected throttling, event timestamps, cautious reporting
Process analysis and optimizer	REQ-09 to REQ-12	Process listing, eligibility checks, confirmation, action results
Network monitor	REQ-13 to REQ-15	Network statistics and configured usage alerts
Profile manager	REQ-16 to REQ-23	Predefined profiles, selection, modification, profile-based decisions
Alerts and logging	REQ-20 to REQ-23	Alerts, recommendations, timestamped logs, recent-event history

4. Architecture Diagram



5. Why this Architecture is Suitable

- **Modularity**: each major feature has a distinct responsibility.
- **Maintainability**: changes to detection rules need not require rewriting the dashboard.
- **Testability**: monitoring, detection, profiles, and optimization can be tested independently.
- **Safety**: potentially disruptive actions pass through confirmation and permission checks.
- **Extensibility**: additional detection rules, monitoring metrics, or performance profiles can be added later.
- **Traceability**: each component maps to identifiable SRS requirements and test cases.